

Logi**Smart**

Defensa TPO · 9 jun 2026

Proceso de Desarrollo de Software · UADE 2026

Primero el negocio. Después, la arquitectura.

LogiSmart es una plataforma SaaS de logística para PyMEs. La recorreremos capa por capa: cada decisión de diseño existe para sostener una necesidad concreta del negocio.

Rosario PresedoLU 1146894

Simón OttatiLU 1155931

Comisión BelgranoUADE · 2026

5

capas

5

atributos

36

patrones

190

tests ✓

EL CLIENTE Y SU PROBLEMA

Una PyME que hoy gestiona su logística a mano

Hoy · fragmentado
Planillas, WhatsApp
y memoria

- Pedidos del e-commerce cargados a mano, uno por uno.
- Rutas armadas «a ojo», sin tráfico ni ventanas horarias.
- El cliente final llama para preguntar «¿dónde está mi paquete?».

→

LogiSmart
Con LogiSmart · centralizado
Una sola plataforma
SaaS multitenant

- Importa pedidos del e-commerce automáticamente.
- Arma rutas óptimas según tráfico real y capacidad de flota.
- El cliente final sigue su envío en un mapa, con ETA real.

Misión: convertir la logística de un centro de costos en una ventaja competitiva. **Visión:** una solución todo-en-uno, económica y fácil de implementar, que una sola plataforma sirva a muchas PyMEs —cada una su propio «tenant».

QUÉ LE PROMETIMOS

Tres casos de uso son el corazón del MVP

CU-01

Importar pedidos

Sincroniza con TiendaNube / MercadoShops y trae las órdenes pendientes — adiós a la carga manual.

CU-03

Planificar rutas óptimas

Analiza pedidos, vehículos y tráfico real, y devuelve la secuencia de paradas más eficiente respetando ventanas horarias.

CU-07

Seguir el envío en vivo

El cliente final entra por un enlace, ve al chofer en un mapa y el ETA calculado según el avance real de la ruta.

Actores internos

Operador Logístico

crea envíos · asigna rutas · gestiona flota · reportes

Admin. PyME / Empresa

crea envíos · consulta reportes

Admin. de Plataforma (LogiSmart)

administra tenants · todos los permisos

Transportista

ejecuta rutas · actualiza posición GPS

Cliente Final

sigue su envío en tiempo real · CU-07

→

Logi**Smart**

SaaS multitenant

→

Sistemas externos

Carriers

DHLFedExUPS

Pagos

PayPalStripe

Mapas

Google Maps · ARHERE Maps · BR

CÓMO DEFINIMOS «BIEN HECHO»

Cinco atributos de calidad: la brújula del diseño

Crítica

Alta

disponibilidad

Si LogiSmart se cae, se frena la operación de la PyME. Objetivo 99,9% de uptime.

→ Aplicación

Alta

Aislamiento

de datos

La info de cada PyME debe ser inaccesible para otros abonados del SaaS.

→ Presentación

Alta

Interoperabilidad

Integración fluida con e-commerce, mapas y pagos heterogéneos, sin contaminar el dominio.

→ Infraestructura

Media-alta

Elasticidad

Adaptar recursos según demanda y región (AR / BR) para optimizar costos.

→ Infra + Dominio

Media

Eficiencia

de respuesta

Cálculo de rutas y consultas de tracking que respondan rápido.

→ Persistencia

Cada patrón sostiene uno de estos atributos — **no «porque sí»**. Y los atributos **compiten**: diseñar es decidir estas tres tensiones.

Seguridad ⇔ Performance

El Controller valida permisos en cada operación (paga latencia); Proxy + caché + Lazy Load la recuperan en Persistencia.

Flexibilidad ⇔ Simplicidad

Asumimos más clases que un condicional — solo donde hay un punto de variación real (tarifas, estados, proveedores).

Anticipar ⇔ Sobre-ingeniería

Protegemos los puntos de variación real; cada abstracción nace de un problema concreto, no de anticipar el futuro.

EL NEGOCIO, MONTADO SOBRE LAS CAPAS

Cada atributo de calidad vive en una capa

flujo de una petición

01

Presentación

Recibe la petición, autoriza y delega. Sin lógica de negocio.

Aislamiento de datos

02

Aplicación

Orquesta los casos de uso entre subsistemas, sin que nada quede a medias.

Alta disponibilidad

03

Dominio · el corazón

Las reglas del negocio. Todo gira en torno a Envío y su ciclo de vida.

Trazabilidad en tiempo real

04

Persistencia

Aísla el almacenamiento del dominio. El dominio nunca toca SQL.

Eficiencia de respuesta

05

Infraestructura

Capa transversal: crea objetos e integra con el mundo externo. La usan todas las demás.

Interoperabilidad

Elasticidad

e-commerce mapas pagos

Argentina Brasil

CAPA 01 · PRESENTACIÓN

Que ninguna PyME vea los datos de otra

Atributo **Aislamiento de datos** · seguridad (alta)

Permisos antes que nada

Controller

Una sola puerta de entrada que valida quién pide qué, y delega.

aplicacion/ · LogiSmartController

Recibe las solicitudes de la interfaz, **valida permisos** e inyecta dependencias por constructor. No calcula ni persiste.

Trade-off: la capa queda deliberadamente delgada — exige disciplina para no filtrar lógica de negocio hacia arriba.

Permisos sin if frágiles

Rol · Polymorphism

El acceso de cada rol resuelto con polimorfismo, no con condicionales.

dominio/ · Rol (enum) · IPermisos

El enum Rol centraliza la matriz de permisos **5x5** y elimina 25 booleanos duplicados. Evita instanceof y switch.

Trade-off: la matriz vive en un único enum; un rol radicalmente nuevo obliga a extenderla — aceptable porque los 5 roles del SaaS son estables.

Multitenant por diseño

El control de acceso vive arriba para que el **dominio quede testeable en aislamiento** — y para que los datos de un tenant nunca crucen hacia otro.

← expandí una tarjeta para ver el fragmento en código

LogiSmartController.java

```
public Envio crearEnvio(IPermisos solicitante, ...) {  
    if (!solicitante.puedeCrearEnvio())  
        throw new SecurityException("Sin permiso para crear envíos");  
}
```

Rol.java

```
OPERADOR(true, true, true, true, false), // puede todo  
CLIENTE (false, false, false, false, false); // mismo método → distinto resultado
```

25 booleanos dispersos, un enum centralizado

25
booleanos
en 5 clases
→
1
enum · fuente
de verdad única

Un permiso nuevo: antes, **5 archivos** a editar. Ahora, **una columna más en el enum**. El cambio se realizó con la batería de tests en verde — sin miedo a romper nada.

Antes — duplicado en 5 clases

```
// AdminEmpresa.java
private boolean puedeCrearEnvio    = true;
private boolean puedeAsignarRuta   = false;
private boolean puedeVerReportes   = true;
// ídem en Transportista, ClienteFinal...
// → 5 clases × 5 permisos = 25 booleans
```

Después — GRASP Expert centralizado

```
// Rol.java – única fuente de verdad
//          crear asig rep flota admin
OPERADOR      (true, true, true, true, false),
ADMIN_EMPRESA (true, false, true, false, false),
ADMIN_PLATAFORMA (true, true, true, true, true),
CLIENTE       (false, false, false, false, false),
TRANSPORTISTA (false, false, false, false, false);
```

CAPA 02 · APLICACIÓN

Orquestrar el caso de uso sin que nada quede a medias

Atributo **Alta disponibilidad** · confiabilidad (crítica)

Una llamada para una operación compleja

Facade

Una interfaz simple sobre cinco subsistemas que se coordinan en orden.

aplicacion/ · ServicioLogisticaFacade

crearEnvio() orquesta inventario, pago, rastreo, notificación y reportes en el orden correcto. El cliente no conoce ese orden.

Trade-off: riesgo de «God Object» — por eso la fachada **solo coordina y delega**, nunca calcula ni valida ella misma.

Fallar barato y temprano

Chain of Responsibility

Validación en cadena que corta en el primer rechazo.

chain/ · CadenaValidadores

5 validadores en orden de costo: datos → inventario → pago → seguridad → capacidad. Corta apenas uno falla (fail-fast).

Trade-off: el orden importa — agregar un validador es una clase nueva, pero hay que ubicarlo bien en la cadena.

Orquesta, no decide

Es la capa que más se apoya en GRASP: **bajo acoplamiento y alta cohesión**. La lógica real siempre baja al dominio.

+ En esta capa también: **Command** · **Mediator** · **Template Method**

CHAIN OF RESPONSIBILITY + VALUE OBJECT + PROTECTED VARIATIONS

Un patrón, una deuda técnica, una corrección

Antes · deuda

```
// Firma frágil: cada campo nuevo
// obliga a modificar TODOS los nodos
boolean validar(Envio envio, Cobro cobro) {
    ...
}
```

Con el quinto validador (Hito 10) la firma creció. Agregar un campo cascadea a toda la cadena — la deuda acumula intereses.

Después · liquidada

```
// ContextoValidacion = Value Object
// Los nodos no cambian cuando el
// contexto crece
boolean validar(ContextoValidacion ctx) {
    ctx.getEnvio(); ctx.getCobro();
}
```

ContextoValidacion encapsula el contexto completo. Los validadores son **Protected Variations**: inmunes a cómo crezca el contexto mañana.

Agregar el quinto validador fue una clase nueva. Liquidar la deuda fue un refactor acotado a un solo punto. **Open/Closed en acción** — la cadena creció sin tocar los nodos existentes.

CAPA 03 · DOMINIO

Envío, **el corazón del negocio**

dominio · entidad

Envío

Builder

se construye paso a paso

Prototype

se clona en lotes

State

evoluciona de estado

Strategy

calcula su costo

Memento

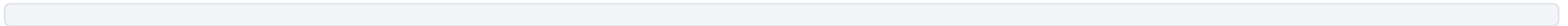
guarda su historial

Observer

notifica sus cambios



Seleccioná un patrón para ver su rol en Envío — o seguí al slide siguiente para ver State, Observer y Memento en detalle.



CAPA 03 · DOMINIO

Que el cliente final siga su envío en tiempo real

Promesa estrella **Trazabilidad** + robustez del envío

Transiciones válidas, siempre

State

El envío evoluciona solo, siguiendo su máquina de estados.

state/ · Confirmado → EnTránsito → EnReparto → Entregado

Cada estado decide sus transiciones válidas; las inválidas se rechazan solas. Sin State, entregar() sería un switch gigante.

Trade-off: más clases que un condicional, pero las reglas quedan explícitas y agregar «EN_ADUANA» es una clase nueva.

Avisar a todos al instante

Observer

Al cambiar de estado, el envío notifica a sus suscriptores.

observer/ · Dashboard · Auditoría · Notificaciones

El cambio CONFIRMADO → EN_TRÁNSITO avisa solo. Agregar «notificar al teléfono» es suscribir un observador, sin tocar Envío.

Trade-off: un observador que falla puede cortar al resto — se aísla cada notificación y, a escala, se haría asíncrona.

Deshacer y auditar

Memento

Una foto inmutable del envío para restaurar o comparar.

memento/ · MementoEnvio · HistorialEnvios

Snapshot de solo lectura (estado, origen, destino, costo, timestamp) sin exponer los campos privados de Envío.

Trade-off: guardar historial cuesta memoria — se conserva solo lo necesario para auditar y reconstruir.

+ El dominio también modela: CompositeStrategyVisitorDecoratorInterpreter

CAPA 04 · PERSISTENCIA

Respuestas rápidas sin que el dominio toque SQL

Atributo **Eficiencia de respuesta** + consistencia

El dominio pide objetos, no filas

Repository + Data Mapper

Acceso uniforme que esconde si el dato viene de memoria o de SQL.

persistencia/ · RepositorioEnvios · EnvioMapper

El Envio se reconstruye vía **Builder**, reutilizando el dominio existente. Cambiar el backend no afecta al negocio.

Trade-off: reutilizar el dominio vs un registro plano introduce desfasaje — mitigado declarando el Composite como única fuente de verdad.

No pegarle 10 veces a la base

Proxy + Lazy Load

Cachea consultas y difiere la carga hasta que se necesita.

proxy/ · ProxyRepositorioEnvios

Consultar el mismo envío 10 veces no debe pegarle 10 veces a la base. Es transparente: el cliente no lo distingue del real.

Trade-off: la caché puede quedar obsoleta — aceptable para lecturas de tracking; se invalida en cada escritura.

Todo o nada

Unit of Work

Agrupar los cambios y los confirma en una transacción atómica.

persistencia/ · UnitOfWork

Confirma todos los cambios juntos o ninguno. Garantiza coherencia ante un fallo a mitad de una operación.

Trade-off: mantener la unidad de trabajo agrega bookkeeping — justificado por la criticidad del dato logístico.

► **Decisión clave:** reutilizar el dominio existente — el mapper traduce solo el subconjunto persistente de cada entidad.

RUP ITERATIVO EN ACCIÓN — HITO 7 → HITO 13

Una decisión de H7 que no sabía lo que hacía por H13

HITO 7 — BUILDER PARA EL DOMINIO

El Builder se diseñó para construir Envio con 12 atributos de forma legible. La motivación era el dominio — no la persistencia.

```
// FabricaDeEnvios.java – H7
Envio envio = new Envio.EnvioBuilder("e-001")
    .origen("BUE").destino("COR")
    .peso(12.5).tipo("EXPRESS")
    .build();
```

HITO 13 — EL MISMO BUILDER, PARA PERSISTENCIA

El Data Mapper necesitaba reconstruir Envio desde la base de datos. El Builder ya existía. **No hubo refactoring.**

```
// EnvioMappersQL.java – H13
Envio envio = new Envio.EnvioBuilder(row.id)
    .origen(row.origen).destino(row.destino)
    .peso(row.peso).tipo(row.tipo)
    .build(); // ← mismo Builder, 6 hitos después
```

Lo sembrado en el Hito 7 se cosechó en el Hito 13. Así funciona el enfoque iterativo: las decisiones de diseño tempranas pagan dividendos iteraciones después — si están bien pensadas.

CAPA 05 · INFRAESTRUCTURA — TRANSVERSAL

Hablar con el mundo externo y operar en dos países

Atributos **Interoperabilidad** + **Elasticidad regional**

Integrar sin contaminar el dominio

Adapter

Traduce APIs externas incompatibles a un contrato propio.

adapter/ · DHL · FedEx · UPS · PayPal · Stripe

Las APIs externas son cajas negras con firmas y unidades distintas. El adapter las lleva a ProveedorEnvio / ProveedorPago.

Trade-off: cada proveedor nuevo es un adapter nuevo — pero el dominio nunca cambia al cambiar de proveedor.

Coherencia por región (AR y BR)

Abstract Factory

Familias de objetos compatibles, nunca piezas mezcladas.

abstractfactory/ · FactoryArgentina · FactoryBrasil

Cada fábrica produce un trío coherente: vehículo + calculador (IVA 21% vs ICMS 12%) + proveedor de mapas. Nunca un auto argentino con cálculo brasileño.

Trade-off: agregar una región es una familia entera nueva — el costo de garantizar coherencia regional.

3 tipos × 4 formatos → 7 clases, no 12

Bridge

Separa qué se reporta del cómo se formatea.

bridge/ · ReporteEnvios · ReporteIngresos · ReporteDesempeno · PDF · Excel · JSON · CSV

Reporte define el contenido; GeneradorReporte define el formato. Agregar XML es 1 clase nueva. Agregar Auditoría es 1 clase nueva. Sin Bridge serían 12 subclases combinadas.

Trade-off: indirección extra — el reporte delega el formateo en lugar de producir la salida directamente.

10.000 envíos, no 30.000 objetos

Flyweight

Comparte ubicaciones inmutables que se repiten muchísimo.

flyweight/ · Ubicacion · FabricaUbicaciones

Origen y destino se repiten («Buenos Aires», «Córdoba»). 10.000 envíos a 3 ciudades = 3 instancias compartidas.

Trade-off: exige inmutabilidad estricta (campos final) para ser seguro de compartir entre hilos.

+ También transversal: **Factory Method** · **Singleton**

Dos países, dos reglas de negocio — nunca mezcladas

FactoryArgentina

- **Vehículo:** Auto
- **Calculador:** IVA 21% sobre base
- **Mapas:** Google Maps Argentina

```
new LogiSmartFactoryArgentina()  
  .crearVehiculo()      // → Auto  
  .crearCalculador()    // → IVA 21%  
  .crearProveedorMapas() // → GoogleMaps AR
```

FactoryBrasil

- **Vehículo:** Moto
- **Calculador:** ICMS 12% sobre base
- **Mapas:** HERE Maps Brasil

```
new LogiSmartFactoryBrasil()  
  .crearVehiculo()      // → Moto  
  .crearCalculador()    // → ICMS 12%  
  .crearProveedorMapas() // → HERE Maps BR
```

Sin el patrón: un auto argentino con cálculo brasileño (ICMS 12%) produce costos incorrectos silenciosamente. No hay error en compilación — solo un número mal.

La Factory garantiza que los objetos de una región son **siempre coherentes entre sí**. Nunca un Auto argentino con HereMaps brasileño.

Composite facilita agregar nodos · Visitor facilita agregar operaciones

Composite — la estructura

```
NodoCentroRegional      // Composite
├─ NodoPuntoEntrega      // Leaf
└─ NodoPuntoEntrega      // Leaf

aceptar(visitor) ← punto de unión
```

- ✓ Nuevo nodo → 1 clase nueva, árbol listo
- ✓ Cliente trata árbol y hoja igual (*transparencia*)
- ✗ Nueva operación → habría que tocar la interfaz

Visitor — las operaciones

```
VisitorCalculoOcupacion
VisitorCostoOperativo
VisitorBusquedaCriticos
  ↓ double dispatch
visitar(NodoCentroRegional nodo)
```

- ✓ Nueva operación → 1 Visitor nuevo, nodos intactos
- ✓ 3 operaciones sumadas sin modificar el árbol
- ✗ Nuevo tipo de nodo → actualizar todos los Visitors

La estructura del árbol de centros es **estable** (regional + puntos de entrega). Las *operaciones* crecieron hito a hito. Por eso Visitor fue la decisión correcta: el árbol absorbió tres nuevas operaciones sin tocar un solo nodo existente.

El flujo de un envío, capa por capa

ARQUITECTURA CONSCIENTE — LAS DECISIONES DE UBICACIÓN

Por qué cada patrón está en esa capa y no en otra

Controller · Rol · IPermisos

Presentación

El check de permisos vive arriba para que el dominio quede **testeable en aislamiento** — sin infraestructura de seguridad.

Facade · Chain of Responsibility

Aplicación

La orquestación del orden de los subsistemas es responsabilidad de la aplicación — **ni el cliente ni el dominio** deben conocer ese orden.

Flyweight · Ubicacion

Infraestructura

El pool de objetos compartidos es una optimización técnica — **no es una regla de negocio**. El dominio no sabe que existe.

Repository · Data Mapper

Persistencia

El dominio pide **objetos**, nunca filas SQL. Si mañana cambia la base de datos, el dominio no cambia.

Abstract Factory

Infraestructura

La creación de objetos regionales es **transversal a todas las capas**. La coherencia AR/BR no es una regla de negocio — es una restricción de configuración.

MADUREZ DE INGENIERÍA — SACRIFICIOS DELIBERADOS Y DEUDA QUE LIQUIDAMOS

Lo que decidimos *no* hacer — y lo que nos obligamos a corregir

Distancia fija de **500 km** en EstrategiaHibrida

→

Sacrificamos variabilidad realista para tener **tests deterministas**. Un test que depende de `Math.random()` no es un test confiable.

Naming del **Visitor** con prefijo `Nodo`

→

Sacrificamos brevedad para evitar **colisión semántica** con el Composite de centros. Dos patrones en el mismo árbol exigen nombres que no se confundan.

Desfasaje: dominio **jerárquico** (Composite) vs. persistencia **plana**

→

Aceptamos el desfasaje estructural y lo neutralizamos declarando el Composite como **única fuente de verdad**. El costo de un mapeo perfecto era mayor al riesgo.

Deuda técnica — los intereses que decidimos no seguir pagando

ContextoValidacion — Value Object en Chain of Responsibility

✓

La Chain original pasaba Envío y Cobro por separado. Al agregar el quinto validador, la firma se volvió frágil — cada nuevo parámetro habría cascadeado a todos los nodos. Creamos ContextoValidacion (Value Object) y aplicamos **Protected Variations**: los validadores no saben cómo crece el contexto.

MODELO DE NEGOCIO — USUARIOS

5 roles, 1 empresa-tenant, 1 suscripción SaaS

Admin. Plataforma

LogiSmart · acceso total

- Crear envíos
- Asignar rutas
- Ver reportes
- Gestionar flota
- **Administrar empresas**

Operador Logístico

PyME · usuario principal

- Crear envíos
- Asignar rutas
- Ver reportes
- Gestionar flota

Admin. Empresa

PyME · dueño / gerente

- Crear envíos
- Ver reportes
- Gestiona la suscripción

Transportista

PyME · chofer

- Ejecuta rutas
- Reporta posición GPS
- Registra entregas
- Sin permisos de gestión (matriz Rol)

Cliente Final

Externo · destinatario

- Sigue su envío en vivo (CU-07)
- Sin acceso al sistema de gestión

Cada PyME es un **tenant** con una **Suscripción** (fee mensual por cantidad de envíos). Los permisos salen de la matriz 5x5 del enum Rol — la misma de la slide 08.

MODELO DE NEGOCIO — FLOTA

3 tipos de vehículo · 2 regiones con reglas distintas

Moto

- **Carga máx.:** 100 kg
- **Costo/km:** \$0.90
- **Default región:** Brasil (Factory)
- **Creación directa:** VehiculoFactory

Auto

- **Carga máx.:** 500 kg
- **Costo/km:** \$1.00
- **Default región:** Argentina (Factory)
- **Creación directa:** VehiculoFactory

Camión

- **Carga máx.:** 5.000 kg
- **Costo/km:** \$1.80
- **Default región:** ninguna
- **Creación:** solo VehiculoFactory

Argentina

- **Vehículo default:** Auto
- **Impuesto:** IVA 21 %
- **Fórmula:** $(\$100 + \text{peso} \times \$15) \times 1.21$
- **Mapas:** Google Maps
- **Ej. 10 kg:** \$302.50

Brasil

- **Vehículo default:** Moto
- **Impuesto:** ICMS 12 %
- **Fórmula:** $(\$80 + \text{peso} \times \$12) \times 1.12$
- **Mapas:** HERE Maps
- **Ej. 10 kg:** R\$224.00

La selección de región ocurre en `LogiSmartApp.crearFactory(region)` — único punto del sistema con nombres de fábricas concretas. El resto del código solo ve `LogiSmartFactory`.

MODELO DE NEGOCIO — ENVÍO

El ciclo de vida completo de un envío

Estado del envío (State)

CONFIRMADO → EN_TRANSITO

→ EN_REPARTO → ENTREGADO

↳ CANCELADO · RETENIDO desde estados intermedios

Tipos de proceso (Template Method)

- **Nacional:** peso × \$100
- **Internacional:** peso × \$250 + \$1.000
- **Urgente:** peso × \$180 + \$500

Validaciones (Chain · 5 pasos)

- 1 · Datos (origen, destino, peso)
- 2 · Inventario (stock)
- 3 · Pago (cobro válido)
- 4 · Seguridad (destino no restringido)
- 5 · Capacidad (vehículo disponible)

Notificaciones (Observer)

- Dashboard operativo en tiempo real
- Email / SMS al cliente final
- Centro de distribución

Estrategias de costo (Strategy · 6)

- **Peso:** kg × \$5
- **Distancia:** km × \$500
- **Urgencia:** \$500 / \$300 / \$100
- **Volumen:** m³ × \$4
- **Fijo:** monto configurable
- **Híbrida:** 40 % dist + 30 % peso + 30 % urg

Servicios adicionales (Decorator)

- **GPS tracking:** +\$50 fijo
- **Seguro:** +15 % sobre total
- El orden de decoradores importa

Prometimos

CU-01 Importar pedidos del e-commerce

CU-03 Planificar rutas óptimas

CU-07 Seguir el envío en vivo

Cumplimos

13

hitos

5

capas

36

patrones

229

clases

190 ✓

tests

Primero el negocio.

Después, la arquitectura.

¿Preguntas?

Rosario Presedo · Simón Ottati — Comisión Belgrano · UADE 2026